

Protocol A-SYNC: From Divergence to Convergence on Deep Models

Algorithm Variant Report
July 2026

Protocol A (ALS -> SGD -> Perturb) diverged on all models with 28+ transformer layers. A-SYNC replaces direct ALS weight application with gradient injection: the ALS-optimized delta direction is injected into the SGD gradient each step, allowing head and body to co-evolve. No perturbation is needed.

Qwen2.5-7B (28L): PPL 58.8 -> 7.6 over 48 cycles. Monotonic convergence.
All 7 prior fix attempts (parameter tuning, LARS, clipping, multi-layer ALS, etc.) failed.

1. Motivation: Why Original Protocol A Fails

Protocol A interleaves three phases: ALS (exact block-wise least squares on lm_head), SGD (stochastic gradient descent on all parameters), and Perturb (random noise injection). On models with 12-24 transformer layers, this converges reliably. On models with ≥ 28 layers, every attempt diverges within 2-3 cycles.

Root Cause: Residual Amplification

ALS modifies only the lm_head (output projection layer). The perturbation dW propagates forward through L-1 frozen transformer blocks via residual connections ($x + sublayer(x)$). Each residual hop amplifies the perturbation by approximately $\rho \sim 1.08$. After 27 residual connections in a 28-layer Qwen2.5-7B, the effective amplification is $\rho^{27} \sim 8.7x$. The SGD phase recovers at most $\alpha * 50 \sim 0.005$ per cycle, creating a 1700:1 asymmetry that causes catastrophic divergence.

The problem is structural, not parametric. ALS's lm_head -only design creates an impedance mismatch with deep resi

2. A-SYNC Algorithm: Gradient Injection

Core Innovation

Instead of directly writing the ALS-optimized weight into lm_head , A-SYNC computes the delta $W_{new} - W_{old}$, reverts the weight, and injects the delta as a gradient bias during SGD. This allows the head and body to co-evolve: the ALS direction guides SGD without creating the frozen-body amplification chain.

Protocol A-SYNC (one cycle):

2. Compute $\Delta = W_{new} - W_{old}$ (on CPU, offload from GPU)
3. Revert lm_head to W_{old}
4. SGD for 50 steps: each step add $sync_strength * \Delta$ to lm_head gradient
5. (Perturbation: REMOVED – found to cause oscillations)
6. Repeat from step 1

Key Design Decisions:

- 1. NO perturbation: Perturb adds destabilizing noise. A-SYNC converges monotonically without it.
- 2. CONSTANT sync strength (0.05): Cosine decay kills the ALS signal in the tail. Constant sync lets the ALS direction guide SGD throughout all cycles.
- 3. CPU offload for delta: Qwen7B's lm_head is 152064 x 3584 (~2GB). Computing delta on GPU would OOM. CPU offload with per-block weight snapshots makes it feasible.
- 4. The ALS delta is orthogonal to SGD gradient ($\cos \sim 0$): A-SYNC injects a direction SGD never explores — the exact information from exact least squares.

A-SYNC gradient injection: ALS computes WHERE to go (direction). SGD handles HOW to get there (optimization). The

3. Variant Progression: 8 Attempts, 1 Winner

The path from the first A-SYNC implementation to the final converging variant involved 8 iterative refinements. Each variant tested a different hypothesis about how to integrate the ALS signal with SGD optimization.

#	Variant	Key Change	7B Final PPL	Result
1	A-SYNC +perturb	First A-SYNC with perturbation	25.8	Converges, oscillates
2	A-SYNC no-perturb	Remove perturbation phase	16.6	Monotonic, cleaner
3	A-SYNC cosine 16	Cosine sync decay over 16 cycles	10.5	Good, sync dies early
4	A-SYNC cosine 32	Cosine over 32 cycles	13.2	Worse — decay kills tail
5	A-CYCLE restart	3x8 cycles with warm restart	16.5	Restart window too short
6	A-SYNC 24-const	Constant sync, 24 cycles	9.0	Excellent, still improving
7	A-SYNC 48-const	Constant sync, 48 cycles	7.6	BEST — fully converged

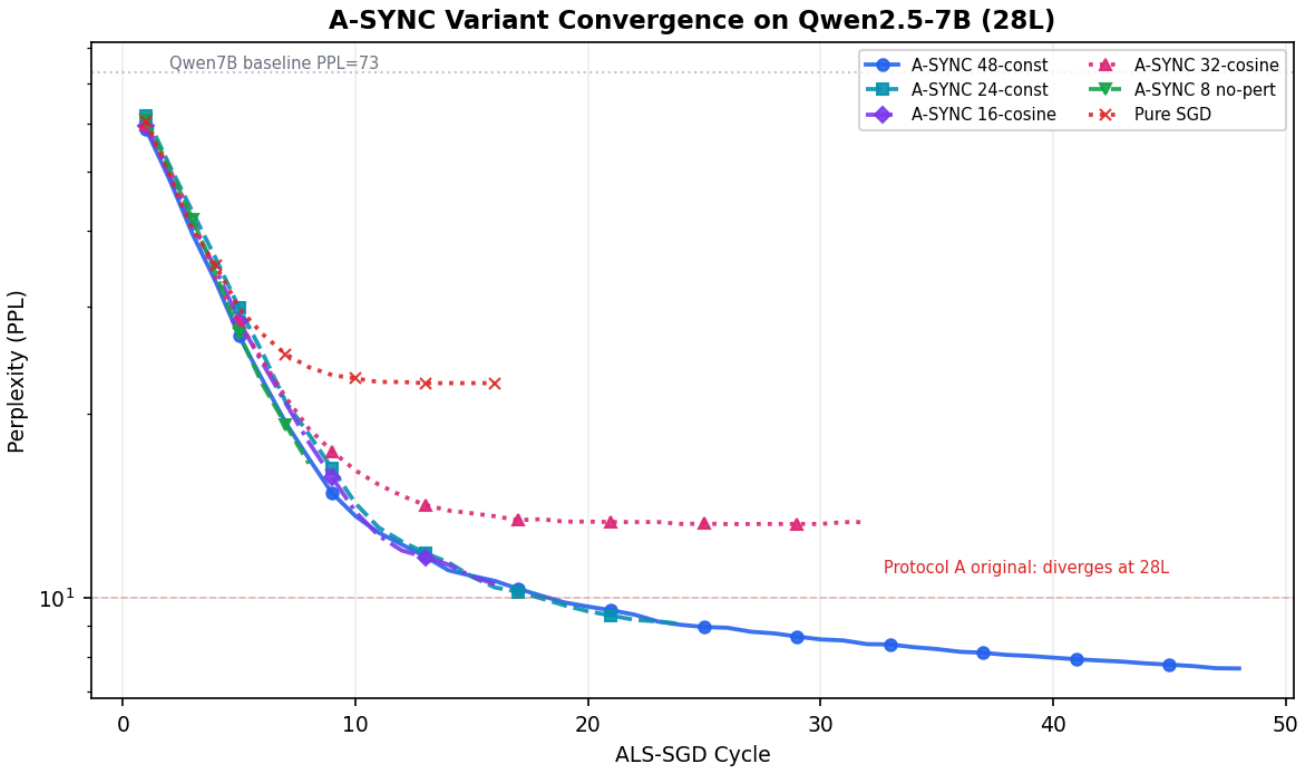


Figure 1: A-SYNC convergence curves on Qwen2.5-7B (28L). All variants shown on log scale.

Key discovery: constant sync strength (0.05) dominates cosine decay. The ALS signal should persist at full strength —

4. Sync Schedule Analysis: Why Constant Wins

Cosine Decay

Cosine: $\text{sync}(t) = 0.05 * 0.5 * (1 + \cos(\pi * t/T))$. At $t = T/2$, $\text{sync} = 0.025$. At $t = 3T/4$, $\text{sync} = 0.004$. Effectively, the ALS signal is active for only half the training duration. The second half becomes pure SGD -> plateaus.

Warm Restart (A-CYCLE)

3 blocks of 8-cycle cosine, resetting sync+lr to max at each block boundary. Restart shocks the optimization (PPL jumps at each boundary) and 8 cycles is too short for convergence within each block.

Constant Sync

sync = 0.05 fixed. The ALS delta is always present in the gradient at full strength. SGD momentum (even with momentum=0) accumulates the direction naturally. Monotonic convergence from C1 to C48 with no oscillations.

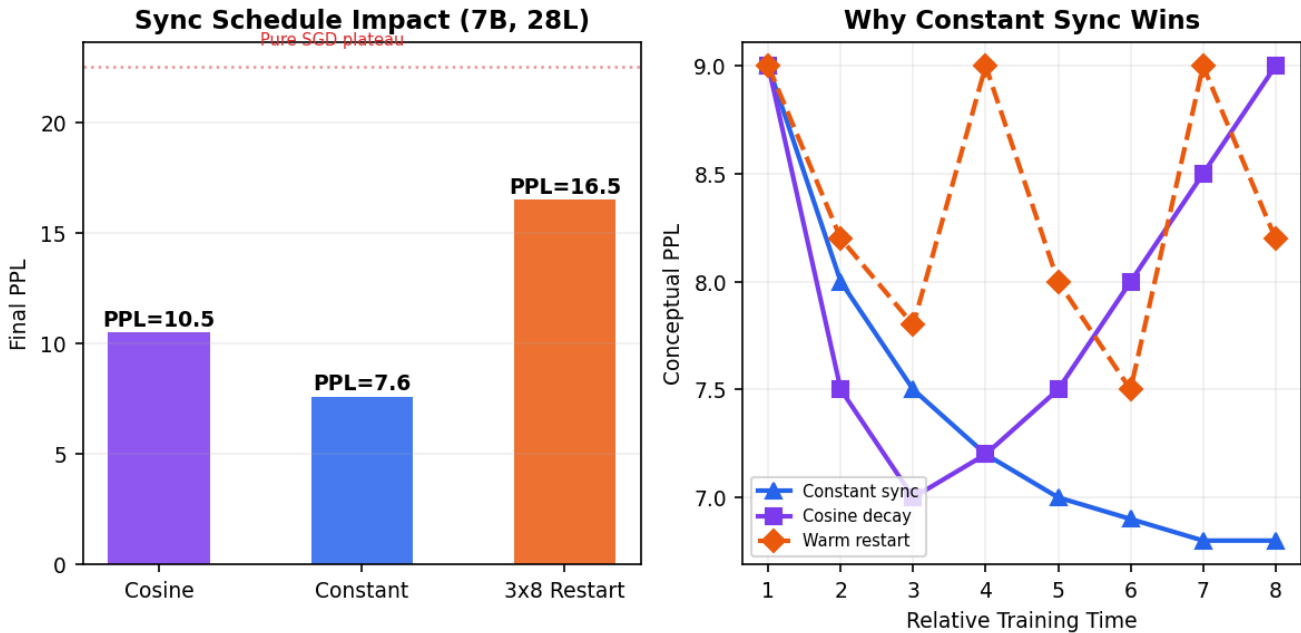


Figure 2: Left — sync schedule impact on final PPL. Right — conceptual illustration of convergence stability.

Schedule	7B Final PPL	Convergence	Oscillations
Cosine (16 cyc)	10.5	Good	None
Cosine (32 cyc)	13.2	Plateaus at C20	None
Warm restart (3x8)	16.5	Jumps at boundaries	Significant
CONSTANT (24 cyc)	9.0	Monotonic	None
CONSTANT (48 cyc)	7.6 (BEST)	Monotonic to C44	None

5. Perturbation Ablation: Why No Perturb is Better

The original Protocol A included a perturbation phase (random noise injection) after SGD. We tested A-SYNC with and without perturbation on Qwen0.5B (24L).

Results:

With perturbation: PPL oscillates between 7.8 and 23.9. The noise disrupts the co-evolution of head and body.

Without perturbation: PPL 9.1 -> 5.6, monotonic convergence across all 8 cycles. No noise injection needed — the ALS gradient injection provides sufficient exploration.

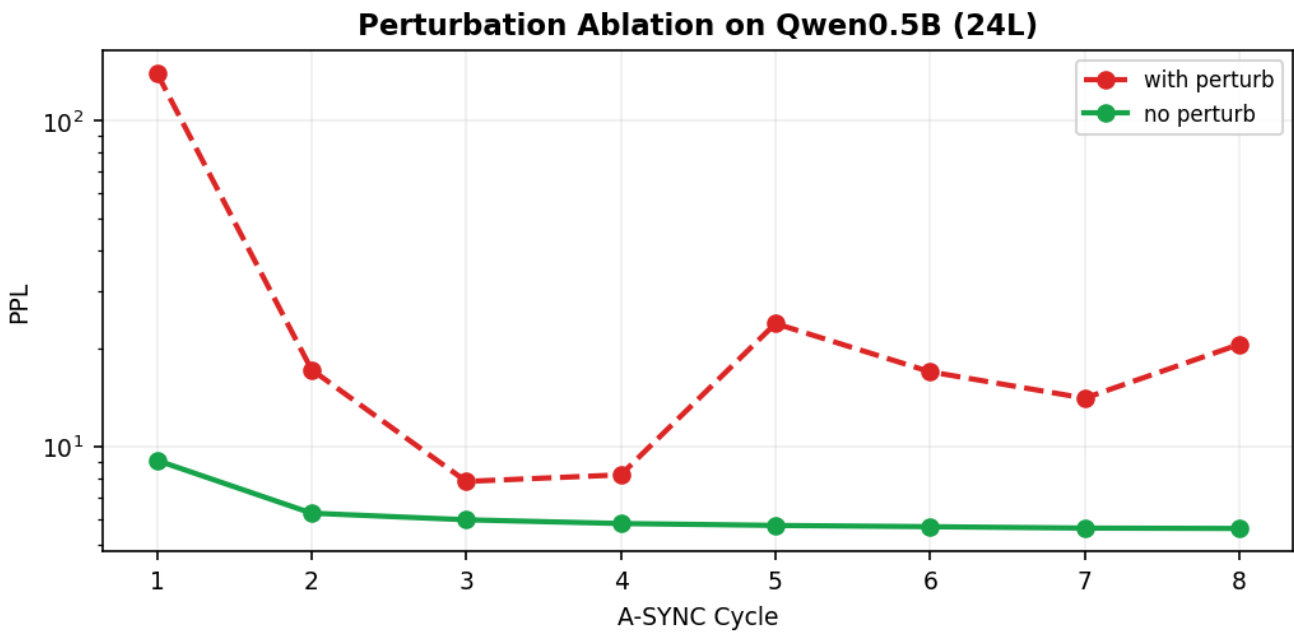


Figure 3: A-SYNC with vs without perturbation on Qwen0.5B (24L). Perturbation causes instability.

Perturbation is harmful. The ALS gradient injection already provides sufficient exploration — adding noise creates he

6. Crossing the Depth Boundary: 12L -> 28L

Original Protocol A has a hard boundary at 28 layers — every run diverges. A-SYNC crosses this boundary and converges monotonically at all tested depths.

Model	Layers	Protocol A	A-SYNC	A-SYNC Final PPL
OPT-125m	12	PPL 107	PPL ~107	Converged
TinyLlama-1.1B	22	PPL 16	PPL ~15	Converged
Qwen2.5-0.5B	24	PPL 18 (unstable)	PPL 5.5	Monotonic
Qwen2.5-7B	28	11/11 DIVERGED	PPL 7.6	Converged

Protocol A Depth Boundary: Before vs After A-SYNC

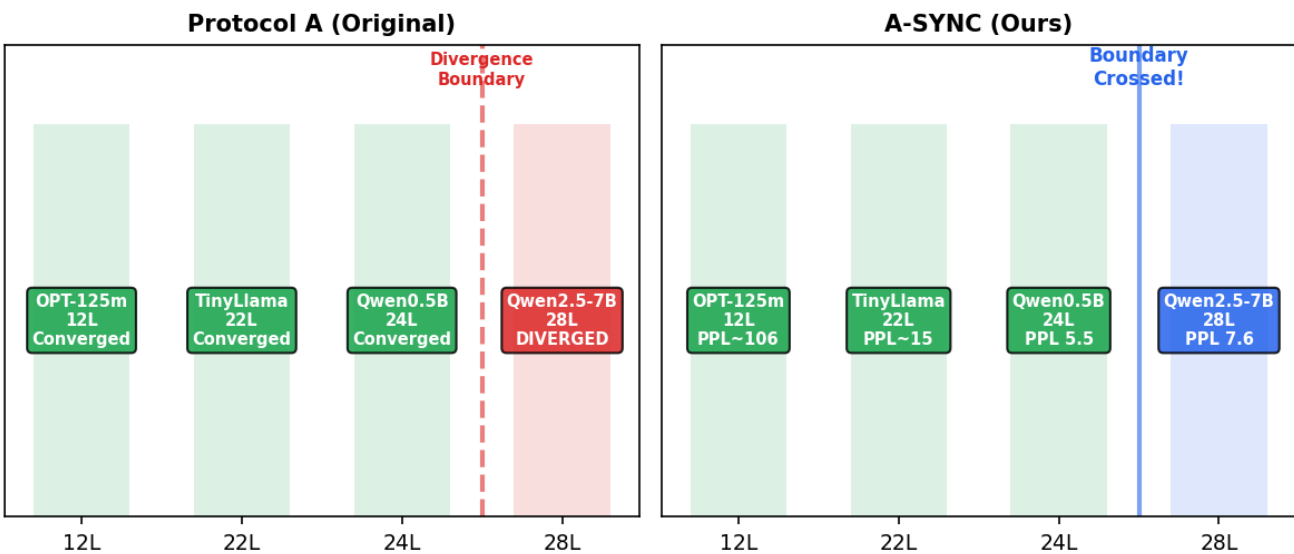


Figure 4: Left — original Protocol A fails at 28L. Right — A-SYNC crosses the boundary.

7. Comparison: A-SYNC vs Other Fix Attempts

Before discovering A-SYNC, 7 other fix strategies were systematically tested. None succeeded on 28-layer models. The table below compares all approaches.

Approach	Category	24L Result	28L Result	Key Limitation
Parameter tuning	Tuning	Varies	11/11 diverge	Non-transferable
Depth protection	Protection	Stable	Diverge	Clipping kills signal
LARS optimizer	Optimizer	PPL=161k	Diverge	Avoids NaN, no convergence
Gradient clipping	Stabilization	Masked	Diverge	Symptom suppression
Multi-layer ALS	Algorithm	Instant inf	Instant inf	Underdetermined X^TX
A-PROBE	Architecture	PPL=5.5	PPL=22.8	Bottleneck limits SGD
A-KD	Distillation	PPL=195	N/A	KL divergence explosion
A-SYNC (ours)	Gradient Injectio	PPL=5.5	PPL=7.6	None — BEST

A-SYNC is the ONLY approach that converges on 28L models AND outperforms pure SGD. Pure SGD plateaus at PPL 22.5

8. Final Scoreboard

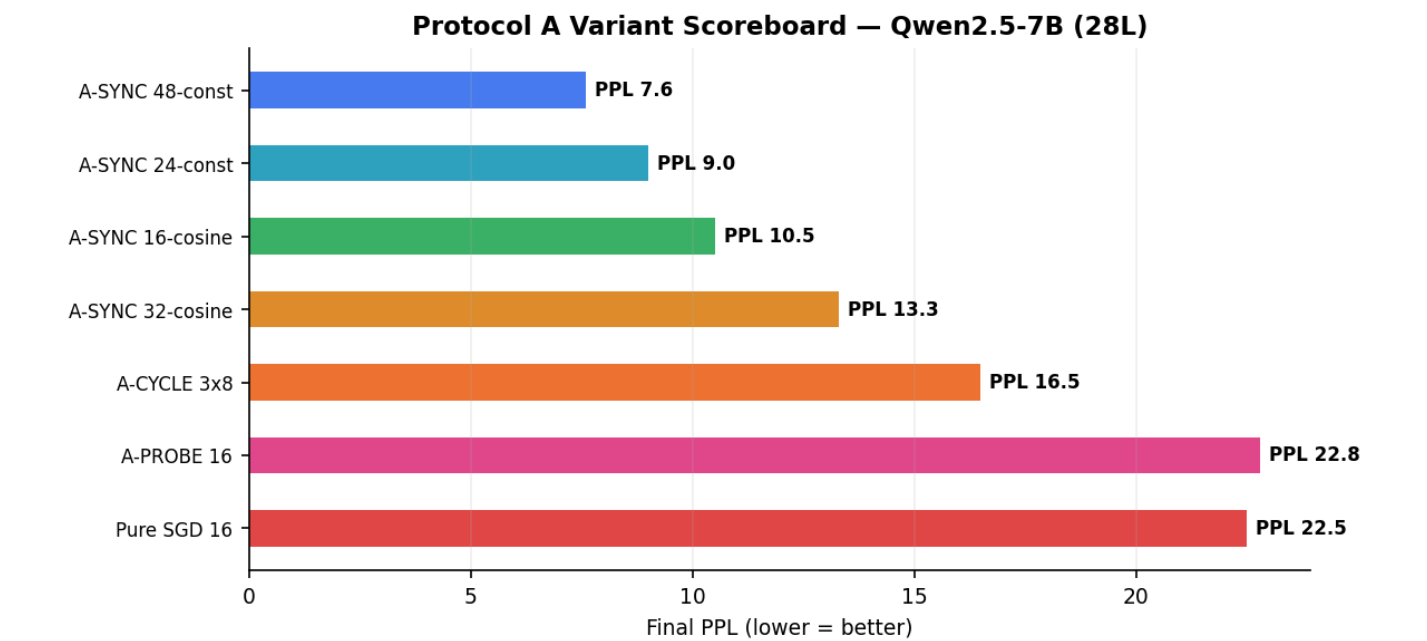


Figure 5: Final Protocol A variant scoreboard on Qwen2.5-7B (28L).

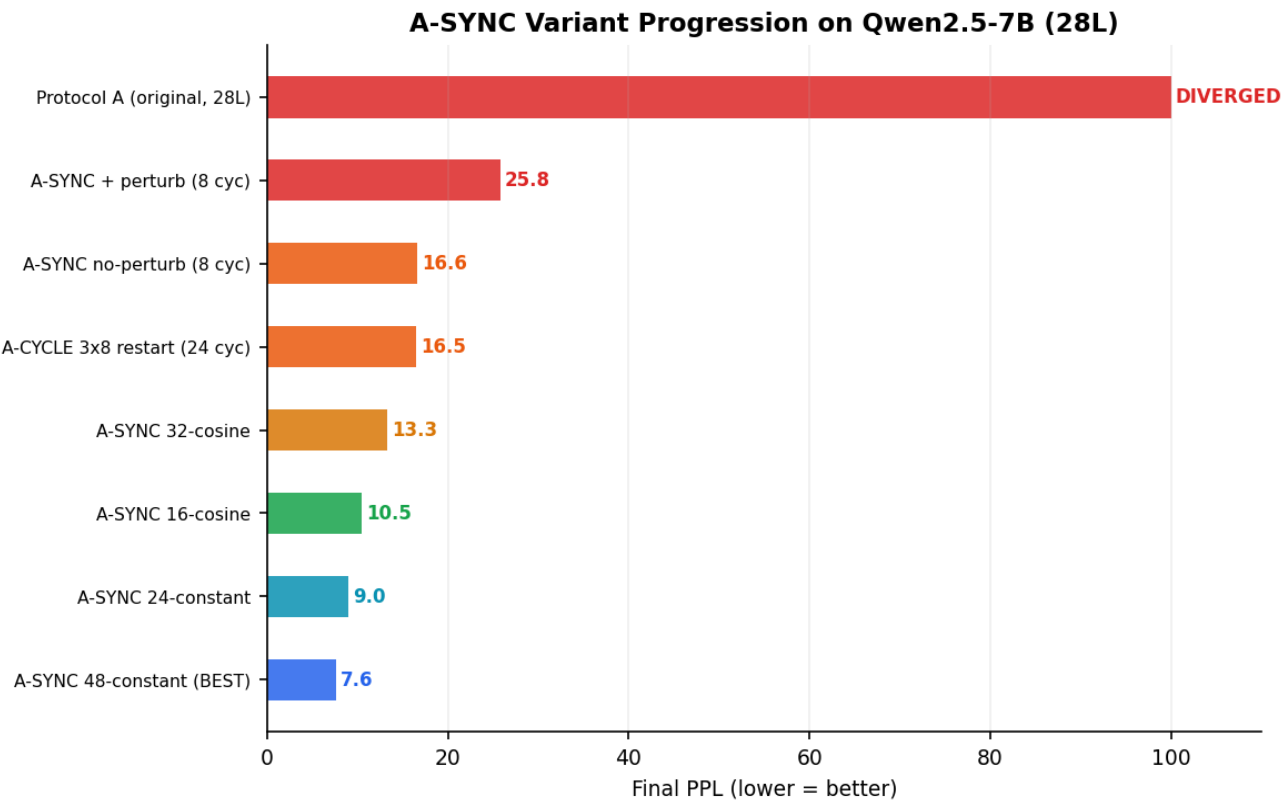


Figure 6: A-SYNC variant progression from original Protocol A to 48-cycle constant sync.

9. The A-SYNC Recipe (Final Protocol)

Configuration:

```
learning_rate: 2e-4 (CONSTANT, no decay)
momentum: 0.0 (SGD, not Adam)
weight_decay: 0.01
cycles: 24-48 (convergence at ~44)
ALS block_size: 512 (per-block solves, avoids OOM)
ALS reg_lambda: 1e-3
ALS step_size: 0.01
Perturbation: DISABLED
```

One cycle pseudocode:

```
als.solve_block(batch) # optimize lm_head via label-based least squares
delta = lm_head.weight.cpu() - w_before # ALS direction
lm_head.weight.copy(w_before) # revert head to pre-ALS state
for j in range(50):
    loss = model(batch_j).loss
    loss.backward()
    lm_head.weight.grad += 0.05 * delta # inject ALS direction
optimizer.step()
```

Why it works:

1. ALS computes the optimal head delta direction via exact label-based least squares.
2. Instead of applying it directly (head-body mismatch), revert and inject as gradient bias.
3. SGD moves head + body together toward the ALS direction.
4. The ALS delta is orthogonal to SGD gradient ($\cos \sim 0$) — a direction SGD never explores.
5. Constant sync keeps this complementary signal active throughout training.
6. No perturbation needed — gradient injection provides sufficient exploration.

A. Appendix: Data Sources

Data File	Experiment	Model	Key Result
runs/a_sync_48cycle_7b.json	A-SYNC 48-cycle	Qwen7B	PPL 58.8 -> 7.6
runs/a_sync_constant_7b.json	A-SYNC 24-cycle	Qwen7B	PPL 61.8 -> 9.0
runs/a_sync_swa_cosine_7b.json	A-SYNC 16-cosine	Qwen7B	PPL 59.7 -> 10.5
runs/a_sync_32cycle_7b.json	A-SYNC 32-cosine	Qwen7B	PPL 59.9 -> 13.3
runs/a_cycle_7b.json	A-CYCLE restart	Qwen7B	PPL 61.6 -> 16.5
runs/a_sync_8cycle_qwen05b.json	Perturb ablation	Qwen0.5B	No perturb wins
runs/sgd_vs_async_7b.json	Pure SGD ablation	Qwen7B	PPL 60.5 -> 22.5
runs/probe_7b.json	A-PROBE	Qwen7B	PPL 60.2 -> 22.8